

WeTravel API Overview

The API for WeTravel is grouped based off functionality that is planned in the MVP. As such it is grouped into categories. These categories are explained below:

1. Authentication Endpoints:

Purpose:

This section handles the account registrations, user logins, and generation of JWT tokens that enable clients to relay back current user credentials.

POST /auth/register - Parses account creation details, hashes given password, creates database User object from it, saves it to the database.

POST /auth/login - validates login request by checking if the user exists and that password matches the one stored inside of the database. Once validation is complete a JWT(json web token) is returned to the client ([react.js](#) app) containing a User ID and the Username of the account being logged into so that the client can relay back and forth to the server who they are.

Relevant Pydantic Validation Models:

FastAPI parses client sent payload for needed attributes to instantiate needed Objects to be used inside of Endpt function:

- UserRegistration
 - email
 - username
 - password

Function: Parses client Payload data and performs Validation for required details for User registration.

- LoginRequest
 - username
 - password

Function: Parses Payload data and performs Validation for required details for login requests.

- TokenResponse
 - access_token
 - token_type

Function: Used to send back a formatted response object back to the client with only relevant information about the token and its type.

Use Cases:

1. User Registration

User enters registration data

→ POST /api/auth/register

- Pydantic validates UserRegistration
- password is hashed
- User ORM object is created
- database stores user
- success response returned

2. User log in

User enters username/password

- POST /api/auth/login
- credentials are verified
- JWT is generated
- JWT returned to client
- client stores JWT

2. User/Profile Endpoints:

Purpose:

This section handles the account registrations, user logins, and generation of JWT tokens that enable clients to relay back current user credentials.

GET /users/me - Retrieves the current user's profile details and sends back a response that includes the user_id, username, email, and posts attached to the account.

GET /users/{user_id} - Achieves the same as the previous endpoint, except that it retrieves another user's profile

Relevant Pydantic Validation Models:

1. UserResponse

- user_id
- username
- email
- Posts

Purpose: Response format Object that filters what is being returned to the client for safety.

2. PublicUserResponse

- user_id
- username
- Posts

Purpose: Same as previous, except email is omitted due to the response being that of another user beside the one logged in.

Use Cases:

1. View my Profile:

Client sends JWT

- GET /api/users/me
- get_current_user() verifies JWT
- user_id extracted from token
- User ORM object retrieved
- UserResponse returned

2. View another's profile

User selects another traveler

- GET /api/users/{user_id}
- database retrieves User by primary key
- PublicUserResponse returned

3. Post/Feed Endpoints:

GET /posts: Retrieves standard travel feed

GET /posts/{post_id} : Retrieves one specific post

GET /users/me/posts: Retrieves current user's posts

GET /users/{user_id}/posts: Retrieves another user's posts

POST /posts : Create new post

DELETE /posts/{post_id} : Delete post owned by current user

Relevant Pydantic Validation Models:

PostCreate:

- caption
- media_url
- location

Purpose: Validates the post data sent by the client.

PostResponse:

- post_id
- user_id
- Caption

-media_url
-location
-created_at

Purpose: Controls the post data returned to the client.

Functionality Notes

GET /posts

- Retrieves latest posts.
- Ordered by `created_at`.
- Current implementation limits results to [20].
- Future versions may add pagination, ranking, and location filtering.

POST /posts

- Requires JWT.
- User ID comes from `current_user.user_id`.
- Client does not send `user_id`.
- Creates SQLAlchemy Post object.
- Saves object to database.

DELETE /posts/{post_id}

- Retrieves the post by primary key.
- Verifies that `post.user_id == current_user.user_id`.
- Returns 403 if the current user does not own the post.

Use Cases

1. Loading standard feed

GET /api/posts

→ database queries Post table

→ newest posts selected

→ list[PostResponse]

→ JSON returned to client

2. Creating a post

PostCreate payload

→ JWT identifies current user

→ Post ORM object created

→ database insert

→ PostResponse returned

3. Delete my post

DELETE /api/posts/{post_id}

→ retrieve Post

→ compare owner to current user

→ delete if authorized

→ 204 No Content

4. Location Endpoints:

Purpose:

This section handles the manipulation of locations items and its associated properties.

GET /locations/{location_id} : Retrieves information about one location

GET /locations/{location_id}/posts : Retrieves all of the posts associated with one location

Relevant pydantic validation models:

LocationResponse

-location_id

-name

-latitude

-longitude

Purpose: represents location information returned from database

Use Cases

View location

User selects location

- GET /api/locations/{location_id}
- Location ORM object retrieved
- LocationResponse returned

View posts from a location

User selects location on map

- GET /api/locations/{location_id}/posts
- retrieve related posts
- list[PostResponse] returned

5. Itinerary Endpoints:

GET /itineraries : Gets all of the itineraries belonging to current user.

POST /itineraries : Creates an Itinerary

GET /itineraries/{itinerary_id} : retrieves one specific Itinerary

Relevant Pydantic validation models:

ItineraryCreate

Name
Description
start_date
end_date
Destinations

Purpose: Create an Itinerary for the current user.

ItineraryResponse

Itinerary_id
User_id
Name
Start_date
End_date
Destinations

Purpose: Response format for details database returns

Use Cases

View my itineraries

JWT identifies current user

- GET /api/itineraries
- retrieve current_user.itineraries
- return list

Create itinerary

ItineraryCreate

- current_user.user_id
- Itinerary ORM object
- database insert
- ItineraryResponse

6. Safety Report Endpoints:

Purpose: Safety reports are retrieved and created based off of individual user and location for the safety of all.

GET /safety-reports : Retrieve list of safety reports

POST /safety-reports: Create safety report

SafetyReportCreate

- report_type
- location
- description

Purpose: Validation of Report details

SafetyReportResponse

- report_id
- user_id
- report_type

- location
- description
- created_at

Purpose: Validates response message details given to client.

Use Cases

View safety reports

GET /api/safety-reports
→ database retrieves SafetyReport rows
→ list[SafetyReportResponse]
→ returned to client

Submit safety report

SafetyReportCreate
→ JWT identifies user
→ SafetyReport ORM object created
→ database insert
→ SafetyReportResponse returned

7. Authentication and Authorization Overview

Which operation needs JWT to validate user?

Operation	Requirement
Register	Public
Login	Public
View standard Feed	Public
View individual Post	Public
View another User	Public/[TBD]

View current User	JWT required
Create Post	JWT required
Delete Post	JWT + ownership required
View Itineraries	JWT required
Create Itinerary	JWT required
Submit Safety Report	JWT require